

Performance testing my TCP pub/sub broker: a journal

This is a write-up of the performance work I did on my [pub/sub broker](#) — a TCP messaging broker with a custom binary protocol, written from scratch in C++. I wanted to know: how fast is it, how consistent is it, and where does it actually break down under load?

What started as “let’s measure throughput” turned into a chain of investigations — a load generator with a subtle measurement bug, a per-message heap allocation storm on both the client and server, a mutex that was quietly stalling my subscriber’s socket reads, and a scheduling experiment that backfired. This document walks through all of it in the order it happened, with real numbers and real graphs from the actual runs.

If you’re building something similar, I hope this is useful both as a set of findings and as an example of *how* to chase down performance problems methodically instead of guessing.

Environment

All numbers in this document come from the same machine, unless stated otherwise:

- **Host OS:** Windows, running WSL2 (Ubuntu)
- **WSL version:** 2.7.14.0, kernel 6.18.33.2-microsoft-standard-WSL2
- **CPU:** 11th Gen Intel Core i5-1135G7 @ 2.40GHz — 4 physical cores / 8 threads, 8 MiB L3 cache
- **Hypervisor:** Microsoft Hyper-V (WSL2 runs Linux inside a lightweight VM)
- **Build:** Release build, `-O3 -DNDEBUG -march=native`

A few things about this environment turned out to matter a lot for the results, and I’d flag them to anyone reading this before trusting any specific number:

- **WSL2 is virtualized.** Every test result includes an extra layer between my code and the physical CPU — Hyper-V’s own scheduler decides when the Linux VM’s virtual CPUs actually get physical CPU time. This turned out to be the most likely explanation for a chunk of the variability I couldn’t fix in code (more on that later).
- **My project directory started out on `/mnt/c/...`**, inside a OneDrive-synced folder. This turned out to be a real source of noise — see the “environment interference” section below. I later moved everything to the native Linux filesystem (`/home/...`), which helped.
- I ran the broker and load generator as separate processes, pinned to separate CPU cores with taskset, to reduce scheduler contention between them.

I’m not claiming these numbers generalize to a production server or bare-metal Linux — they’re internally consistent (same machine, same method) and useful for *comparing before and after* a given code change, which is what I actually used them for.

The tools I built

Before any of the results below make sense, here's what I was actually running:

- **loadgen** — a load generator that connects to the broker as both a publisher and a subscriber. It publishes messages on a fixed schedule, tags each one with a unique ID, and records timestamps for when a message was *intended* to be sent, when it *actually* was sent, and when the subscriber *received* it. From that, it computes latency (receive time - send time) per message and reports percentiles.
- **Percentile reporting**: P50, P90, P99, P999 (99.9th), and max — not just an average. More on why below.
- **A per-sample latency log** — later in the investigation, I added the ability to dump every (message index, latency) pair above a threshold, which turned out to be the single most useful diagnostic I built.

The test shape

Every result in this document comes from the same simple topology: **one publisher, one topic, one subscriber** — a single publisher connection sending to a single topic, with a single subscriber connection receiving from it. There's no fan-out (multiple subscribers on one topic) and no multiplexing (multiple publisher/subscriber connections) anywhere in this investigation — that's deliberately out of scope here and would be a natural next round of testing. Keeping it to one publisher and one subscriber meant every latency number reflects a single message's round trip through the broker, with nothing else competing for the broker's attention — which made it much easier to isolate cause and effect as I worked through each fix.

A quick note on percentiles, for anyone unfamiliar

An average latency hides more than it tells you. If 99 requests take 1ms and one takes 500ms, the average is ~6ms — technically correct, completely misleading. Percentiles fix this: **P99 latency of 400us** means "99% of requests were faster than 400us, and 1% were slower." At real traffic volumes, that "1%" isn't a rounding error — at 5,000 messages/second, P99 means **50 slow requests every single second, continuously**. That's why P999 (99.9th percentile) matters too — it tells you how bad the *rare* case gets, which is often where real problems hide.

1. A measurement bug before any real testing

My first version of the load generator sent one publish request, waited for the broker's acknowledgment, then sent the next. This is called a **closed-loop** design, and it has a well-known problem: if the broker ever stalls, the load generator just waits patiently instead of continuing to send at the intended rate. This means slow periods get "hidden" — you record one slow sample instead of the dozens of samples that *should* have queued up behind it. This effect is called **coordinated omission**, and it makes tail latency look artificially good.

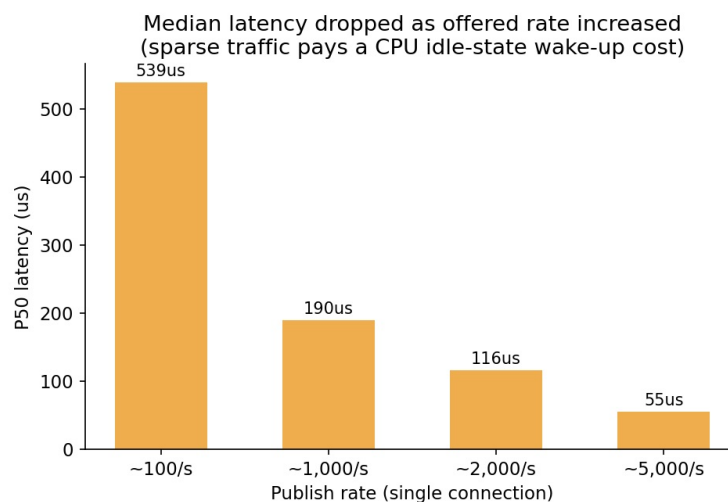
The fix was to make publishing **non-blocking** — the load generator no longer waits for the broker's acknowledgment before scheduling the next send; it fires on a fixed clock instead.

This single fix immediately changed the picture. With the blocking version, at a fixed low rate, latency looked flat and boring. Once publishing was non-blocking and I could push the rate up, a completely different, more honest picture emerged — including the discovery in Section 3 below, which a closed-loop test would have hidden.

Lesson: if your load generator waits for a response before sending the next request, your tail-latency numbers are lying to you by construction, not just by chance.

2. Sparse traffic is slower than busy traffic (and that's not a bug)

Once I could vary the publish rate cleanly, I found something that looked backwards at first: **latency went down as the rate went up**.



Rate vs P50 latency

At ~100 messages/second, median latency was around 540us. At ~5,000/second, it dropped to around 55us — roughly a 10x improvement, just from sending *more* traffic. This isn't a queueing effect (5,000/sec was nowhere near this broker's capacity) — it's the opposite: **sparse traffic gives the CPU time to go idle between messages**, and modern CPUs aggressively drop into low-power sleep states and downclock when there's nothing to do. Waking back up and ramping frequency back up costs real, measurable time — and at low rates, nearly every message pays that wake-up tax. At high rates, the CPU core stays busy and warm, so each message is genuinely cheaper to process.

Lesson: if you're benchmarking something and sparse traffic looks *slower* than busy traffic, check CPU idle-state behavior before assuming something's wrong with your code.

3. The environment was part of the system under test

Early on, I saw wildly inconsistent results between otherwise-identical runs — the same test, run twice in a row, giving very different tail latencies. Two environmental factors turned out to matter:

1. **My project lived on /mnt/c/..., inside a OneDrive-synced**

folder. Every file access there goes through a slower Windows-filesystem bridge, and — more importantly — rebuilding a binary in a OneDrive-watched folder tends to trigger background syncing and antivirus scanning right when I was trying to measure something. I moved the whole project to the native Linux filesystem (/home/. . .), which removed a big source of “why is the first run after a rebuild always slower” noise.

2. **Laptop power state and thermal throttling.** Running many tests back-to-back can shift CPU frequency scaling behavior over time. I didn’t fully control for this, but I stayed aware of it when comparing runs across a long session.

Lesson: on a laptop, “the code” and “the environment” are not cleanly separable — profile where you build and run matters as much as what you build.

4. Eliminating hot-path heap allocation (client and server)

With the environment quieter, the next big signal was a persistently heavy tail — P999 and max latency values in the tens of milliseconds, recurring across runs, even though P50/P90/P99 looked fine. I traced this to **unreserved, growable buffers being allocated fresh on every single message**, on both sides of the connection:

- **Client side (loadgen):** the wire-frame buffer for each publish was a `std::vector<uint8_t>` built via repeated `push_back/insert` calls with no `reserve()` — meaning every message paid for several heap reallocations, most of them cheap, but occasionally one would hit a slow allocator path.
- **Server side (BrokerServer::handlePublish):** the delivery path for each subscriber built up **three separate growable vectors with two extra copies** per delivery — and since this runs once per subscriber per publish, the cost scales with fan-out, not just message count.

The fix in both places was the same idea: **compute the exact final frame size up front, allocate one correctly-sized buffer, and encode directly into it** — no growth, no intermediate copies.

This was a meaningful, repeatable improvement to the typical case (P50–P99), though — as the next chapter shows — it wasn’t the whole story for the worst-case tail.

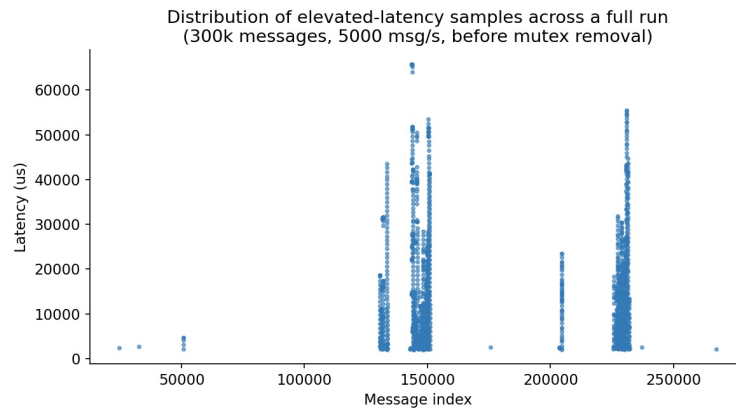
Lesson: on a hot path handling thousands of messages per second, “small” per-message allocations add up, and the rare slow allocator path is exactly the kind of thing that produces occasional large tail-latency spikes.

5. Finding the real root cause: a backlog that drains in lockstep

Even after the allocation fixes, P999 and max latency were still occasionally very large — tens of milliseconds — and highly inconsistent between otherwise-identical runs. At this point I stopped trusting percentiles alone and built a **per-sample latency log**: for every message above a threshold, log its index and latency.

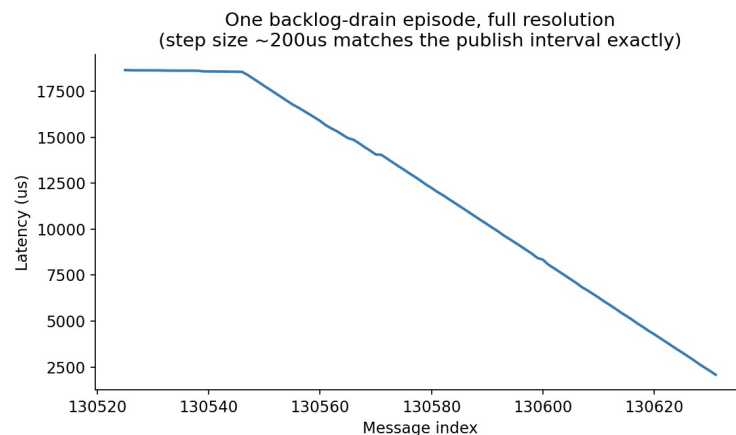
This turned out to be the single most useful thing I built. The first thing it showed was that these elevated samples weren’t scattered randomly through the run — they were dense in the middle third of it,

and largely absent elsewhere:



Distribution of elevated samples across a full run

Zooming into one of those episodes at full resolution revealed a very specific, repeating shape:



One FIFO decay episode

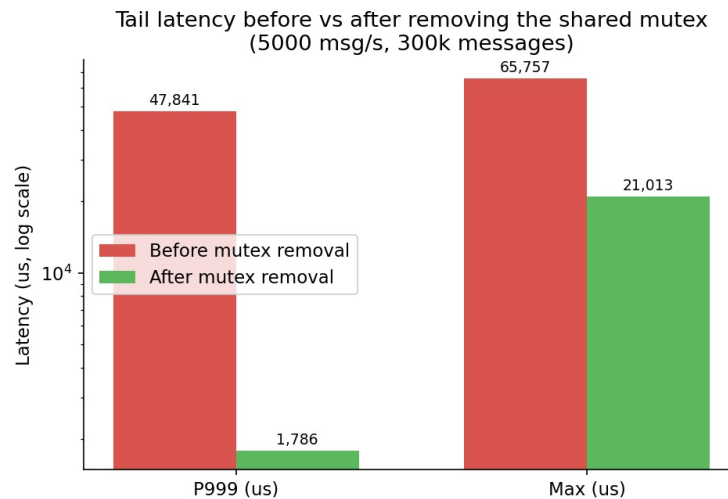
This is one episode from a real run — latency starts around 18,600us and decays almost perfectly linearly, in steps of about 200us, down to normal. That 200us step size isn't a coincidence — **it's exactly my publish interval**. That's the signature of a backlog: something briefly stalled, a queue of undelivered messages built up, and once whatever stalled recovered, the backlog drained in strict first-in-first-out order — each message in the queue exactly one publish-interval closer to the front than the one before it.

Tracing the code, I found the cause: my load generator's subscriber-side socket read loop and its message-processing handler were **not decoupled** — the same thread that reads bytes off the socket also runs the receive-timestamp-recording handler inline, before looping back to read more. And that handler was contending on a mutex (`g_timestamps_mutex`) that the publisher thread was *also* locking on every single message, up to 5,000 times a second on each side. If the subscriber's reader thread ever lost that race, it couldn't get back to reading the socket — bytes queued up, the broker's own per-connection send queue backed up behind it, and a backlog formed.

Lesson: percentiles tell you *that* something's wrong; only looking at raw, ordered samples tells you *what*. If your tail latency is unstable and mysterious, log individual samples with enough context (index, timestamp) to see their shape, not just their distribution.

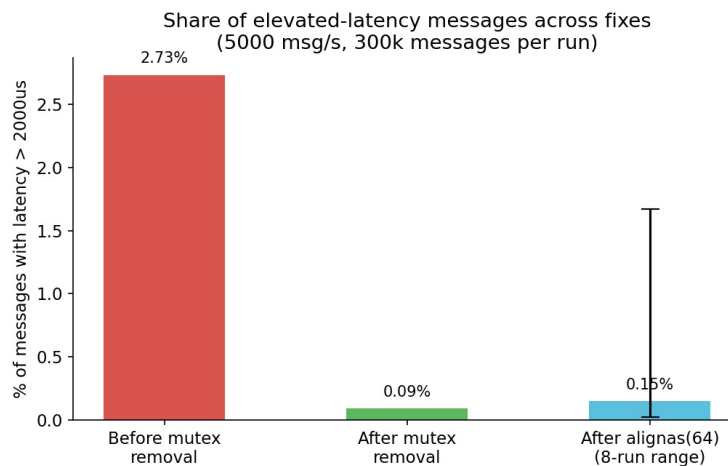
6. Removing the mutex: the biggest single win

The fix was straightforward once the cause was clear: the publisher and subscriber threads only ever write to **different array indices** (a message can't be received before it's sent), so the mutex was protecting against a race that couldn't actually happen. I removed it entirely.



Before vs after removing the mutex

Metric	Before	After	Change
P999 latency	47,841 us	1,786 us	~27x lower
Max latency	65,757 us	21,013 us	~3x lower
Messages > 2000us	8,199 / 300,000 (2.73%)	265 / 300,000 (0.09%)	~31x fewer



Elevated sample percentage across fixes

This was, by a wide margin, the single biggest improvement in the whole investigation. It also confirmed the theory from Section 5 directly — the backlog-drain episodes shrank dramatically in both frequency and severity, and the ones that remained were the same shape, just smaller.

Lesson: a lock that looks “safe but harmless” can be the actual root cause of a serious performance problem, especially if it sits between two threads that both need to stay responsive to I/O. Always ask

whether a lock is protecting a *real* race before assuming it's just overhead.

7. False sharing: a smaller, still-real win

Removing the mutex didn't fully explain the remaining tail, so I checked for **false sharing** — a subtler problem where two threads write to *different* variables that happen to sit on the same CPU cache line, causing constant cache-coherency traffic between cores even with no logical contention at all. My publisher and subscriber threads write to nearby indices in the same timestamp array, which made this a real risk even after the lock was gone.

I aligned the timestamp struct to a 64-byte cache line boundary (`alignas(64)`), giving each entry its own exclusive cache line. Across a batch of repeated runs, the typical-case latency consistently improved:

Metric	Before <code>alignas(64)</code>	After <code>alignas(64)</code>
P50	~54-81 us	~46-70 us
P90	~102-258 us	~89-167 us
P99	~282-19,732 us	mostly ~184-724 us

The rare, large backlog episodes from Section 5 didn't fully go away — they remained bursty and unpredictable, which was itself an important signal (see below).

Lesson: removing a lock doesn't automatically remove cache contention between the same two threads — check for false sharing separately, especially when threads write to nearby memory at high frequency.

8. A scheduling experiment that backfired (and why that's still a useful result)

By this point, the remaining tail latency was highly inconsistent between runs — sometimes 64 elevated messages out of 300,000, sometimes over 5,000, with no code change in between. That pattern (same code, wildly different outcomes) pointed toward something outside my application — most likely the OS or hypervisor occasionally delaying my subscriber's reader thread.

I tested this directly using `chrt -f 50` to run the whole process under `SCHED_FIFO` real-time scheduling, hoping to make the reader thread less likely to be preempted. The result was a clean, repeatable **regression**:

P50 (6 runs)	
With <code>chrt -f 50</code>	82-103 us (all runs)
Without	51-60 us (all runs)

The ranges didn't even overlap — every real-time run was slower than every normal run. The reason: `chrt` elevated *every* thread in the process, not just the one I cared about, onto real-time scheduling on the same two pinned cores. `SCHED_FIFO` threads don't get time-sliced the way normal threads do, so I'd removed the fairness mechanism that was previously making sure my publisher and subscriber threads each got a timely turn — trading one contention problem for another.

Lesson: a plausible-sounding fix is still an experiment, not a guarantee — and a negative result, cleanly measured, is a legitimate and useful outcome. I didn't pursue a more targeted version (elevating only the reader thread) because the blunt version already showed no upside worth chasing further.

Summary: what actually moved the needle

Fix	Effect	Confidence
Non-blocking publish (fixing coordinated omission)	Made tail-latency measurements honest in the first place	High — structural
Moving off /mnt/c / OneDrive	Removed a source of build-triggered noise	High
Reserving/pre-sizing buffers (client + server)	Meaningful, repeatable drop in typical-case latency	High
Removing the shared mutex	~27x lower P999, ~31x fewer elevated messages	High — directly confirmed
alignas(64) (false sharing fix)	Modest, consistent improvement in P50-P99	Medium-high
chrt -f 50 real-time scheduling	Made things worse; abandoned	High (negative result)

What's left is a small, bursty tail that doesn't respond to further application-level tuning — locks, memory layout, and thread priority have all been tried. The most likely remaining explanation is Hyper-V's own scheduling of the WSL2 VM's virtual CPUs, which is outside what application code can control. That's a reasonable place to stop for this round of investigation — the two confirmed wins (mutex removal and cache alignment) accounted for the large majority of the improvement, and the remaining variance is now well-characterized rather than mysterious.

Methodology notes, for anyone repeating this kind of investigation

A few practices that made this investigation actually converge, rather than chasing noise indefinitely:

- **Always report P50/P90/P99/P999/max together, never a single number.** They tell different stories, and a fix that helps one can be neutral or harmful to another (see Section 7 vs Section 8).
- **Repeat every test 3+ times before trusting a result.** Several apparent “regressions” in this investigation turned out to be normal run-to-run variance once measured properly.
- **Change one variable at a time**, and when comparing two versions of the code, build and run both back-to-back in the same session to control for environmental drift.
- **When percentiles get confusing, log raw samples.** The backlog-drain discovery in Section 5 was invisible in percentile summaries alone and obvious the moment I looked at ordered, timestamped samples.
- **A negative result is still a result.** The chrt experiment didn't help, but it ruled out a hypothesis cleanly and is documented here

for exactly that reason.